



Today's agenda

↳ Prefix and Suffix String

↳ LPS of a given string

↳ LPS array of a given string

↳ Problems



AlgoPrep



e) Given a String of Size N .

Prefix Strings: All the substrings starting from 0th idn.

Suffix Strings: " " " ending at last idn.

ex: a b d b

Prefix Strings

a

a b

a b d

a b d b

Suffix Strings

b

d b

b d b

a b d b



Q) Given S_n , LPS of the string.

length of longest prefix which is also a suffix.

Note: other than entire string

(longest proper prefix which is also proper suffix)

Ex: $S = b d a b d$

Prefix

b

b d

b d a

b d a b

b d ~~a~~ b d

Suffix

d

b d $\rightarrow 2$

a b d

d a b d

b d ~~a~~ b d

$S = S_0 S_1 S_2 S_3 S_4$

S_0

$S_0 S_1$

$S_0 S_1 S_2$

$S_0 S_1 S_2 S_3$

S_4

$S_3 S_4$

$S_2 S_3 S_4$

$S_1 S_2 S_3 S_4$

iter:

1

2

3

4

$\vdots$

$n-1$

T.C: $\frac{n + (n-1)}{2}$

$\frac{(n-1) + n}{2} \approx O(n^2)$



Q) Given S_n , return $LPS[n]$

Ex: $s[7]: a a b a a b a$
 $LPS[7]: 0 1 0 1 2 3 4$

$LPS[i] =$ LPS value of substring $\{0, \dots, i\}$

$i:3 \rightarrow$ LPS of $aaba$

Proper Prefix	Proper Suffix
a	a
aa	ba
aab	aba

$LPS[3] = 1$

$i:5 \rightarrow$ LPS of $aabaab$

Proper Prefix	Proper Suffix
a	b
aa	ab
aab	aab
aaba	baab
aabaa	abaab

$LPS[5] = 3$

T.C: $N * N^2 = O(N^3)$

int[] LPS (String s)

// already written
in $O(N)$

expected T.C: $O(N) \rightarrow$ we will cover this in
2nd class of string.

}



Q) Search for a given Pattern P in Text T .
↳ KMP

ex: $T_n = b c a c d a \rightarrow \text{True}$
 $P_k = a c d a$

Idea 1

$T_n = b c a c d a$
 $P_k = a c d a$

T.C: $O(n \cdot k) = k \cdot n/2 \rightarrow O(n^2)$

Idea 2

$T_n = b c a c d a$
 $P_k = a c d a$

$S: \quad \overset{P_k}{\text{a c d a}} \quad \overset{T_n}{\text{b c a c d a}}$
 $LPS[i]: \quad 0 \ 0 \ 0 \ 1 \quad 0 \ 0 \ 1 \ 2 \ 3 \ 4$

↳ if $LPS[i] = P.length$ ^k $\rightarrow \text{return true}$.
T.C: $O(n+k)$

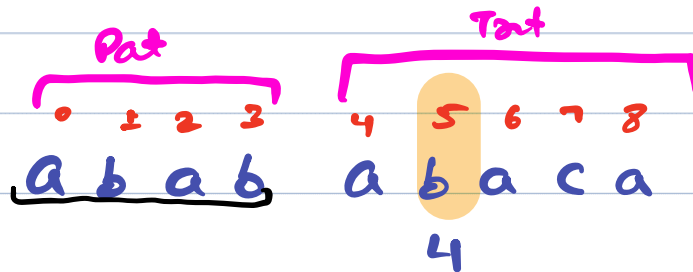


edge case

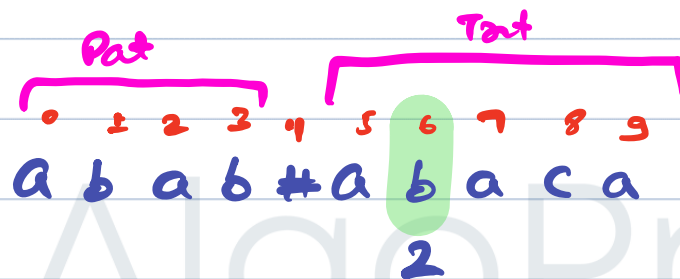
T_n : a b a c a $\rightarrow$ false

P_n : a b a b

#S:
LPS[]:



#S:
LPS[]:



a b a b # a b



a

b

a b

a b

a b a

a b

a b a b

b # a b

a b a b #

a b # a b

a b a b # a

b a b # a b



Steps:

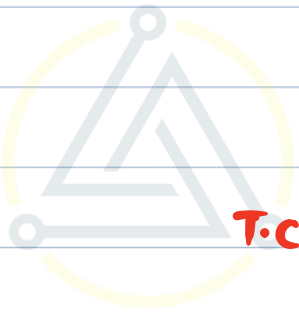
1. $S \leftarrow Pat + \text{"\#"} + \text{text} \rightarrow O(N+K)$

2. Calculate LPS[] of S[] $\rightarrow O(N+K)$

3. if (LPS[i] == Pat.length) { return true; }
;

if none of the idn equals Pat.length
return false.

T.C: $O(N+K)$



AlgoPrep



Q) no. of Substrings of given Pattern P in Text T .

Ex: T_n : ^{0 1 2 3} a a a a $\rightarrow$ ans = 3
 P_k : a a

// idea 1

S : ^{P_k} ^{T_n}
^{0 1 2 3 4 5 6}
a a # a a a a
LPS[]: 0 1 0 1 2 2 2

$\hookrightarrow$ ans = $0 + 1 + 1 + 1 = 3$

$\rightarrow$ Count of P .length in your LPS array of S .

T.C: $O(N+K)$



Q) Given a string S , min characters to be added at the start of string to make entire string Palindrome.

Ex: $dca d a c d \rightarrow ans = 2$

Ex: $dca a b b a a c d \rightarrow ans = 3$

Ex: $a b a d a a d a b a \rightarrow ans = 4$

Idea 1

$S =$

$a b c b a d e b$

$\rightarrow O(N)$
isPalindrome

$str[0-7]$

No

$str[0-6]$

No

$str[0-5]$

No

$str[0-4]$

yes $\rightarrow ans = N - 1 - i$

T.C: $N * O(N) = O(N^2)$



1/idea2

Palind. String

reversed

a b c b a $\rightarrow$ a b c b a

a b c b a d $\rightarrow$ d a b c b a

a b c b a d e $\rightarrow$ e d a b c b a

$\rightarrow$ S = ^{0 1 2 3 4 5 6 7}
a b c b a d e f

S1: a b c b a d e f # f e d a b c b a

LPS of S1 = 5

$\rightarrow$ LPS of whole string = ^{length of} longest possible palindrome starting from 0th idm.

$\hookrightarrow$ ans = length - LPS of whole string
8 - 5 = 3

T.C: $O(2N) \approx O(N)$



→ String S

S = "Hello"

S = S + "e" → $O(N)$

→ if add N strings one by one.
↳ $O(N^2)$

ex: String s: " "

```
for (int i = 0; i < n; i++) {  
    s = s + "1";  
}
```

→ $O(N^2)$

→ String Builder

↳ StringBuilder sb = new StringBuilder();

sb = "Hello";
sb.append("e");

→ $O(1)$

↳ String s = "Hello";

↳ StringBuilder sb = new StringBuilder(s);

$O(N)$ [

```
for (int i = 0; i < n; i++) {  
    sb.append("1");  
}
```

String s1 = sb.toString();

→ $O(N)$



AlgoPrep